

DSA STUDY BLUEPRINT SERIES

31. Two Sum Find Duplicate Hashing Problems

Checking Complements and Duplicate Elements using Hash Maps

Section 05 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Hashing:** Storing elements in key-value sets to enable constant-time lookups.
- **Duplicates:** Store elements in a hash set; duplicate exists if element was seen.

Memory & Execution Blueprint

Value:
5 → [Hash Function]
→

Index Bucket:
2

C++ Implementation Reference

Standard code layout and syntax

```
bool hasDuplicate(vector& nums) {  
    unordered_set s;  
    for (int x : nums) {  
        if (s.count(x)) return true;  
        s.insert(x);  
    }  
    return false;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Average Access	$O(1)$ time, $O(N)$ space
Worst Case Access	$O(N)$ (Hash collisions)

Key Practices & Gotchas

- **Important:** Prefer `unordered_set` over `set` for faster $O(1)$ operations.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace check complement lookups in a hash map for `Two Sum` on `[2, 7, 11]` (target = 9):

Element Scan	Complement: <code>target - x</code>	Hash Map Search	Map Insertion
2	$9 - 2 = 7$	Not found	Insert 2 mapping index 0
7	$9 - 7 = 2$	Found 2 (at index 0!)	Return indices [0, 1]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Prefix Sum Math:** Resolve sub-array queries using math transformations. Complement hash lookups count occurrences instantly.
- **Hash Collisions:** Handled via separate chaining (lists link) or open addressing (linear index probing).
- **Related LeetCode Problems:** #560 Subarray Sum Equals K, #1 Two Sum, #49 Group Anagrams.